

acilis_leaf_gate_alt_distill_qat.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Reconnect T4

Files

Connecting to a runtime to enable file browsing.

ACLIS Leaf Gate — Alternative training (Distill + Field-aug + EMA + QAT)

This notebook is an **A/B alternative** to `acilis_leaf_gate_train_tflite.ipynb`.

	Baseline (current)	This notebook (alt)
Student arch	TinyLeafGate (~29K)	Same TinyLeafGate (fair size comparison)
Teacher	none (from scratch)	MobileNetV3-Small ImageNet → leaf task, then KD
Loss	weighted CE	CE + KL distillation + label smoothing
Augments	crop / flip / jitter / erase	+ blur, JPEG, harsh lighting (camera sim)
Mix	none	MixUp
Optim	Adam 2-phase	AdamW + warmup-cosine (single schedule)
Checkpoint	last best val	EMA weights
Quant	PTQ only	QAT then full-INT8 TFLite
Compare	—	Side-by-side vs baseline TFLite on clean test + camera-stress test

Goal: same MCU flash budget, hopefully better *real-world* / camera-like robustness. Deploy artifact:

`leaf_gate_output_alt/acilis_leaf_gate_96x_alt_full_int8.tflite`

Keep the baseline TFLite on Drive so Section 8 can load both and print a comparison table.

0 — Mount Drive & install packages

```
1 from google.colab import drive
   drive.mount('/content/drive')

Mounted at /content/drive

1 # Same stack as baseline notebook
   !pip install -q tensorflow==2.20.0
   !pip install -q onnx==1.17.0
   !pip install -q onnx-simplifier onnxruntime
   !pip install -q flatbuffers torch torchvision
   !pip install -q scikit-learn matplotlib

16.0/16.0 MB 97.5 MB/s eta 0:00:00
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
19.2/19.2 MB 91.8 MB/s eta 0:00:00
2.7/2.7 MB 85.2 MB/s eta 0:00:00
Building wheel for onnx-simplifier (pyproject.toml) ... done

1 !pip install -q onnx2tf==1.25.2 --no-deps
   !pip install -q sng4onnx
   !pip install -q onnx-graphsurgeon --index-url https://pypi.ngc.nvidia.com
   !pip install -q simple-onnx-processing-tools
   !pip install -q nvidia-pyindex

145.6/145.6 kB 6.0 MB/s eta 0:00:00
440.0/440.0 kB 25.2 MB/s eta 0:00:00
42.1/42.1 kB 3.1 MB/s eta 0:00:00
Preparing metadata (setup.py) ... done
Building wheel for nvidia-pyindex (setup.py) ... done

1 import tensorflow as tf
   import torch

print('✓ TensorFlow :', tf.__version__)
print('✓ PyTorch :', torch.__version__)
print('✓ CUDA :', torch.cuda.is_available())

✓ TensorFlow : 2.20.0
✓ PyTorch : 2.11.0+cu128
✓ CUDA : True
```

1 — Config

Outputs go to a **separate Drive folder** so the baseline checkpoint / TFLite is never overwritten.

```
1 import os
   from pathlib import Path

DRIVE_ROOT = '/content/drive/MyDrive'

PREBUILT_ZIP = f'{DRIVE_ROOT}/leaf_noleaf_dataset.zip'
DATASET_DIR = '/content/leaf_noleaf_dataset'

# Separate from baseline leaf_gate_output/
OUTPUT_DIR = f'{DRIVE_ROOT}/leaf_gate_output_alt'
SAVE_PATH = os.path.join(OUTPUT_DIR, 'acilis_leaf_gate_96x_alt.pth')
EMA_PATH = os.path.join(OUTPUT_DIR, 'acilis_leaf_gate_96x_alt_ema.pth')
TEACHER_PATH = os.path.join(OUTPUT_DIR, 'leaf_gate_teacher_mnv3.pth')
CHECKPOINT_PATH = os.path.join(OUTPUT_DIR, 'acilis_leaf_gate_96x_alt_checkpoint.pth')
TFLITE_PATH = os.path.join(OUTPUT_DIR, 'acilis_leaf_gate_96x_alt_full_int8.tflite')

# Baseline TFLite for side-by-side comparison (from current notebook)
BASELINE_TFLITE = f'{DRIVE_ROOT}/leaf_gate_output/acilis_leaf_gate_96x_full_int8.tflite'

IMAGE_SIZE = 96
NUM_CLASSES = 2
CLASSES = ['leaf', 'not leaf'] # ImageFolder alpha order
BATCH_SIZE = 64
TARGET_ACC = 0.92

# — Alt recipe hyperparams —
TEACHER_EPOCHS = 8 # fine-tune MobileNetV3-Small on leaf task
STUDENT_EPOCHS = 40 # single cosine schedule (vs baseline 5+25)
LR_TEACHER = 3e-4
LR_STUDENT = 1e-3
WEIGHT_DECAY = 5e-4
WARMUP_EPOCHS = 3
KD_TEMPERATURE = 4.0
KD_ALPHA = 0.7 # weight on soft KD loss (rest on hard CE)
LABEL_SMOOTH = 0.05
MIXUP_ALPHA = 0.2
EMA_DECAY = 0.999
QAT_EPOCHS = 5 # fake-quant fine-tune before PTQ export
PATIENCE = 12

SKIP_IF_DATASET_EXISTS = True
```

```

os.makedirs(OUTPUT_DIR, exist_ok=True)

print('✅ Alt config ready')
print(f' DATASET DIR      : {DATASET_DIR}')
print(f' OUTPUT DIR       : {OUTPUT_DIR}')
print(f' TFLITE PATH        : {TFLITE_PATH}')
print(f' BASELINE TFLITE    : {BASELINE_TFLITE}')
print(f' Student epochs     : {STUDENT_EPOCHS}  KD α={KD_ALPHA}  T={KD_TEMPERATURE}')
print(f' MixUp α={MIXUP_ALPHA}  EMA={EMA_DECAY}  QAT epochs={QAT_EPOCHS}')

```

```

✅ Alt config ready
DATASET DIR      : /content/leaf_notleaf_dataset
OUTPUT DIR       : /content/drive/MyDrive/leaf_gate_output_alt
TFLITE PATH      : /content/drive/MyDrive/leaf_gate_output_alt/aclis_leaf_gate_96x_alt_full_int8.tflite
BASELINE TFLITE  : /content/drive/MyDrive/leaf_gate_output/aclis_leaf_gate_96x_full_int8.tflite
Student epochs   : 40  KD α=0.7  T=4.0
MixUp α=0.2  EMA=0.999  QAT epochs=5

```

2 — Load Kaggle leaf / not-leaf dataset

Same normalization as baseline (Leaf → leaf, Non_Leaf → not_leaf).

```

import os
import shutil
import time
import zipfile
from pathlib import Path

print('➤ Dataset cell started', flush=True)

CLASS_ALIASES = {
    'leaf': 'leaf', 'Leaf': 'leaf', 'LEAF': 'leaf',
    'not leaf': 'not_leaf', 'Non_Leaf': 'not_leaf', 'non leaf': 'not_leaf',
    'NonLeaf': 'not_leaf', 'noleaf': 'not_leaf', 'NoLeaf': 'not_leaf',
    'non-leaf': 'not_leaf',
}

def dataset_ready(root):
    for split in ('train', 'val', 'test'):
        for cls in ('leaf', 'not leaf'):
            d = os.path.join(root, split, cls)
            if not os.path.isdir(d) or not any(Path(d).iterdir()):
                return False
    return True

def summarize(root):
    for split in ('train', 'val', 'test'):
        n_leaf = len(os.listdir(os.path.join(root, split, 'leaf')))
        n_not = len(os.listdir(os.path.join(root, split, 'not_leaf')))
        print(f' [{split}] leaf={n_leaf} not_leaf={n_not}', flush=True)

def find_split_root(extract_root):
    for root, dirs, _ in os.walk(extract_root):
        if 'train' not in dirs:
            continue
        children = set(os.listdir(os.path.join(root, 'train')))
        if 'leaf' in children and 'not_leaf' in children:
            return root
        if any(c in children for c in ('Leaf', 'Non_Leaf', 'NonLeaf', 'noleaf')):
            return root
    return None

def normalize_dataset(src_root, dst_root):
    if os.path.isdir(dst_root):
        shutil.rmtree(dst_root)
    for split in ('train', 'val', 'test'):
        split_src = os.path.join(src_root, split)
        if not os.path.isdir(split_src):
            raise FileNotFoundError(f'Missing split folder: {split_src}')
        for name in os.listdir(split_src):
            src_cls = os.path.join(split_src, name)
            if not os.path.isdir(src_cls):
                continue
            canon = CLASS_ALIASES.get(name)
            if canon is None:
                key = name.lower().replace('-', '_').replace(' ', '_')
                if key == 'leaf':
                    canon = 'leaf'
                elif 'non' in key or 'no_leaf' in key or key == 'noleaf':
                    canon = 'not_leaf'
                else:
                    print(f' ⚠ skipping unknown class folder: {split}/{name}', flush=True)
                    continue
            dst_cls = os.path.join(dst_root, split, canon)
            os.makedirs(dst_cls, exist_ok=True)
            for fn in os.listdir(src_cls):
                s = os.path.join(src_cls, fn)
                d = os.path.join(dst_cls, fn)
                if os.path.isfile(s):
                    shutil.move(s, d)
            if not dataset_ready(dst_root):
                raise RuntimeError(f'Normalization failed – check folders under {dst_root}')

if SKIP_IF_DATASET_EXISTS and dataset_ready(DATASET_DIR):
    print(f'✅ Using existing {DATASET_DIR}', flush=True)
    summarize(DATASET_DIR)
else:
    if not os.path.isfile(PREBUILT_ZIP):
        raise FileNotFoundError(
            f'Zip not found: {PREBUILT_ZIP}\n'
            f'Upload leaf_notleaf_dataset.zip to Google Drive root (MyDrive).'
        )
    print(f'✅ Found zip: {PREBUILT_ZIP}', flush=True)
    t0 = time.time()
    local_zip = '/content/leaf_notleaf_dataset.zip'
    if not os.path.isfile(local_zip) or os.path.getsize(local_zip) != os.path.getsize(PREBUILT_ZIP):
        print(' Copying zip to /content...', flush=True)
        shutil.copy2(PREBUILT_ZIP, local_zip)
    extract_tmp = '/content/leaf_notleaf_extract'
    if os.path.isdir(extract_tmp):
        shutil.rmtree(extract_tmp)
    os.makedirs(extract_tmp, exist_ok=True)
    print(' Unzipping...', flush=True)
    with zipfile.ZipFile(local_zip, 'r') as zf:
        zf.extractall(extract_tmp)
    src_root = find_split_root(extract_tmp)
    if src_root is None:
        raise RuntimeError(f'Could not find train/Leaf or train/leaf in {extract_tmp}')
    print(f' Detected dataset root: {src_root}', flush=True)
    normalize_dataset(src_root, DATASET_DIR)
    shutil.rmtree(extract_tmp, ignore_errors=True)
    print(f'✅ Dataset ready at {DATASET_DIR} in {time.time()-t0:.0f}s', flush=True)
    summarize(DATASET_DIR)

from torchvision.datasets import ImageFolder
probe = ImageFolder(os.path.join(DATASET_DIR, 'train'))
print('ImageFolder classes:', probe.classes)

```

```
print(ImageFolder.classes) # probe classes
assert probe.classes == CLASSES, f'Expected {CLASSES}, got {probe.classes}'
print('✅ Class index OK: leaf=0, not_leaf=1', flush=True)
```

```
▶ Dataset cell started
✅ Found zip: /content/drive/MyDrive/leaf_noleaf_dataset.zip
Copying zip to /content...
Unzipping...
Detected dataset root: /content/leaf_noleaf_extract/Leaf v NonLeaf
✅ Dataset ready at /content/leaf_noleaf_dataset in 82s
[train] leaf=9065 not leaf=8066
[val] leaf=1942 not leaf=1728
[test] leaf=1943 not leaf=1730
ImageFolder classes: ['leaf', 'not leaf']
✅ Class index OK: leaf=0, not_leaf=1
```

3 — Models: TinyLeafGate student + MobileNetV3-Small teacher

Student matches baseline topology so flash size stays comparable. Teacher is ImageNet-pretrained and only used during training.

```
print('▶ Model cell started', flush=True)

import copy
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import models

print(f' torch {torch.__version__} cuda={torch.cuda.is_available()}', flush=True)

class DepthwiseSeparable(nn.Module):
    def __init__(self, cin, cout, stride=1):
        super().__init__()
        self.dw = nn.Conv2d(cin, cin, 3, stride=stride, padding=1, groups=cin, bias=False)
        self.bn1 = nn.BatchNorm2d(cin)
        self.pw = nn.Conv2d(cin, cout, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(cout)

    def forward(self, x):
        x = F.relu6(self.bn1(self.dw(x)))
        x = F.relu6(self.bn2(self.pw(x)))
        return x

class TinyLeafGate(nn.Module):
    """Identical topology to baseline – isolates the training recipe as the variable."""
    def __init__(self, num_classes=2):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv2d(3, 32, 3, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(32),
            nn.ReLU6(inplace=True),
        )
        self.blocks = nn.Sequential(
            DepthwiseSeparable(32, 48, stride=2),
            DepthwiseSeparable(48, 64, stride=2),
            DepthwiseSeparable(64, 96, stride=2),
            DepthwiseSeparable(96, 128, stride=1),
        )
        self.head = nn.Sequential(
            nn.AdaptiveAvgPool2d(1),
            nn.Flatten(),
            nn.Dropout(0.2),
            nn.Linear(128, num_classes),
        )

    def forward(self, x):
        x = self.stem(x)
        x = self.blocks(x)
        return self.head(x)

def build_teacher(num_classes=2):
    """MobileNetV3-Small, ImageNet weights, classifier replaced for leaf/not_leaf."""
    try:
        weights = models.MobileNet_V3_Small_Weights.IMAGENET1K_V1
        m = models.mobilenet_v3_small(weights=weights)
    except Exception:
        m = models.mobilenet_v3_small(pretrained=True)
    in_f = m.classifier[-1].in_features
    m.classifier[-1] = nn.Linear(in_f, num_classes)
    return m

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
student = TinyLeafGate(NUM_CLASSES).to(DEVICE)
teacher = build_teacher(NUM_CLASSES).to(DEVICE)

n_params = sum(p.numel() for p in student.parameters())
print('=' * 65)
print('ALT recipe – TinyLeafGate student + MobileNetV3-Small teacher')
print('=' * 65)
print(f'Device           : {DEVICE}')
print(f'Student params    : {n_params:,}')
print(f'Teacher params     : {sum(p.numel() for p in teacher.parameters()):,}')
print(f'Input              : {IMAGE_SIZE}x{IMAGE_SIZE}')
print('=' * 65)
```

```
▶ Model cell started
torch 2.11.0+cu128 cuda=True
Downloading: "https://download.pytorch.org/models/mobilenet_v3_small-047dcff4.pth" to /root/.cache/torch/hub/checkpoints/mobilenet_v3_small-047dcff4.pth
100% |#####| 9.83M/9.83M [00:00<00:00, 137MB/s]
ALT recipe – TinyLeafGate student + MobileNetV3-Small teacher
=====
Device           : cuda
Student params    : 27,538
Teacher params    : 1,519,906
Input             : 96x96
=====
```

4 — Dataloaders (field / camera-hardened augments)

Baseline uses geometric + color jitter. Alt adds Gaussian blur, JPEG compression, and stronger lighting swings to approximate OV2640 / outdoor capture. Val/test stay full-frame Resize(96) (firmware-matched).

```
import io
import random
import numpy as np
from PIL import Image, ImageFilter
from torch.utils.data import DataLoader, WeightedRandomSampler
from torchvision import transforms
from torchvision.datasets import ImageFolder

class SafeImageFolder(ImageFolder):
    def __getitem__(self, index):
        # for offset in range(5):
```

```

        for offset in range(5):
            idx = (index + offset) % len(self.samples)
            path, target = self.samples[idx]
            try:
                img = Image.open(path).convert('RGB')
                if self.transform:
                    img = self.transform(img)
                return img, target
            except Exception:
                continue
        return torch.zeros(3, IMAGE_SIZE, IMAGE_SIZE), 0

class RandomGaussianBlur:
    def __init__(self, p=0.35, radius=(0.3, 1.8)):
        self.p = p
        self.radius = radius
    def __call__(self, img):
        if random.random() > self.p:
            return img
        r = random.uniform(*self.radius)
        return img.filter(ImageFilter.GaussianBlur(radius=r))

class RandomJPEG:
    """Simulate camera / MCU JPEG artifacts."""
    def __init__(self, p=0.4, quality=(35, 90)):
        self.p = p
        self.quality = quality
    def __call__(self, img):
        if random.random() > self.p:
            return img
        q = random.randint(*self.quality)
        buf = io.BytesIO()
        img.save(buf, format='JPEG', quality=q)
        buf.seek(0)
        return Image.open(buf).convert('RGB')

IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD = [0.229, 0.224, 0.225]

train_transform = transforms.Compose([
    transforms.RandomResizedCrop(IMAGE_SIZE, scale=(0.4, 1.0), ratio=(0.85, 1.15)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomVerticalFlip(p=0.3),
    transforms.RandomRotation(degrees=35),
    transforms.ColorJitter(brightness=0.55, contrast=0.55, saturation=0.4, hue=0.12),
    transforms.RandomAffine(degrees=0, translate=(0.12, 0.12), scale=(0.85, 1.15)),
    RandomGaussianBlur(p=0.35),
    RandomJPEG(p=0.4),
    transforms.ToTensor(),
    transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
    transforms.RandomErasing(p=0.2, scale=(0.02, 0.12)),
])

class FixedCameraStress:
    """Deterministic field degradation so baseline vs alt see identical inputs."""
    def __init__(self):
        self.resize = transforms.Resize((IMAGE_SIZE, IMAGE_SIZE))
        self.to_tensor = transforms.ToTensor()
        self.norm = transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD)

    def __call__(self, img):
        img = self.resize(img)
        # mild blur
        img = img.filter(ImageFilter.GaussianBlur(radius=0.9))
        # slightly darker / lower contrast (outdoor underexposure proxy)
        arr = np.asarray(img).astype(np.float32)
        arr = (arr - 127.5) * 0.82 + 127.5 - 12.0
        arr = np.clip(arr, 0, 255).astype(np.uint8)
        img = Image.fromarray(arr)
        # fixed mid-quality JPEG
        buf = io.BytesIO()
        img.save(buf, format='JPEG', quality=55)
        buf.seek(0)
        img = Image.open(buf).convert('RGB')
        x = self.to_tensor(img)
        return self.norm(x)

stress_transform = FixedCameraStress()

val_transform = transforms.Compose([
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
])

train_dataset = SafeImageFolder(os.path.join(DATASET_DIR, 'train'), transform=train_transform)
val_dataset = SafeImageFolder(os.path.join(DATASET_DIR, 'val'), transform=val_transform)
test_dataset = SafeImageFolder(os.path.join(DATASET_DIR, 'test'), transform=val_transform)
# Separate folder instance so stress transform does not touch clean test
stress_dataset = SafeImageFolder(os.path.join(DATASET_DIR, 'test'), transform=stress_transform)

assert train_dataset.classes == CLASSES

counts = [0] * NUM_CLASSES
for _, y in train_dataset.samples:
    counts[y] += 1
print('Train counts:', dict(zip(CLASSES, counts)))

weights = [1.0 / counts[y] for _, y in train_dataset.samples]
sampler = WeightedRandomSampler(weights, num_samples=len(weights), replacement=True)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, sampler=sampler,
                           num_workers=2, pin_memory=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False,
                        num_workers=2, pin_memory=True)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False,
                          num_workers=2, pin_memory=True)
stress_loader = DataLoader(stress_dataset, batch_size=BATCH_SIZE, shuffle=False,
                           num_workers=2, pin_memory=True)

class_weights = torch.tensor(
    [sum(counts) / (NUM_CLASSES * c) for c in counts],
    dtype=torch.float32, device=DEVICE,
)
print('Class weights:', {c: float(w) for c, w in zip(CLASSES, class_weights)})
print('✅ Loaders ready (train + clean val/test + camera-stress test)')

```

Train counts: {'leaf': 9065, 'not leaf': 8066}
 Class weights: {'leaf': 0.9448979496955872, 'not leaf': 1.0619266033172607}
 ✅ Loaders ready (train + clean val/test + camera-stress test)

5a — Train teacher (MobileNetV3-Small)

Short fine-tune on the leaf task. Soft logits become the distillation target for the student.

```

print('► Teacher training started', flush=True)

import math

def evaluate(model, loader, criterion):
    model.eval()
    total_loss, correct, total = 0.0, 0, 0
    class_correct = [0] * NUM_CLASSES
    class_total = [0] * NUM_CLASSES
    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(DEVICE), labels.to(DEVICE)
            outputs = model(images)
            loss = criterion(outputs, labels)
            total_loss += loss.item() * len(images)
            preds = outputs.argmax(1)
            correct += (preds == labels).sum().item()
            total += len(images)
            for p, l in zip(preds, labels):
                class_total[l.item()] += 1
                class_correct[l.item()] += int(p.item() == l.item())
    per_class = {
        CLASSES[i]: (class_correct[i] / class_total[i] if class_total[i] else 0.0)
        for i in range(NUM_CLASSES)
    }
    return total_loss / total, correct / total, per_class

teacher_crit = nn.CrossEntropyLoss(weight=class_weights, label_smoothing=LABEL_SMOOTH)
opt_t = torch.optim.AdamW(teacher.parameters(), lr=LR_TEACHER, weight_decay=WEIGHT_DECAY)
sched_t = torch.optim.lr_scheduler.CosineAnnealingLR(opt_t, T_max=TEACHER_EPOCHS, eta_min=1e-6)

best_t_acc = 0.0
print('=' * 65)
print(f'Teacher fine-tune epochs={TEACHER_EPOCHS} lr={LR_TEACHER}')
print('=' * 65)

for epoch in range(TEACHER_EPOCHS):
    teacher.train()
    total_loss, correct, total = 0.0, 0, 0
    for images, labels in train_loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        opt_t.zero_grad()
        outputs = teacher(images)
        loss = teacher_crit(outputs, labels)
        loss.backward()
        opt_t.step()
        total_loss += loss.item() * len(images)
        correct += (outputs.argmax(1) == labels).sum().item()
        total += len(images)
    sched_t.step()
    tr_acc = correct / total
    _, va_acc, per_class = evaluate(teacher, val_loader, teacher_crit)
    marker = ''
    if va_acc > best_t_acc:
        best_t_acc = va_acc
        torch.save(teacher.state_dict(), TEACHER_PATH)
        marker = ' - saved'
    print(f' Epoch (epoch+1:3d) / {TEACHER_EPOCHS} | train (tr_acc:.3f) | val (va_acc:.3f) | '
          f'leaf={per_class["leaf"]:.3f} not_leaf={per_class["not_leaf"]:.3f} {marker}')

teacher.load_state_dict(torch.load(TEACHER_PATH, map_location=DEVICE))
teacher.eval()
for p in teacher.parameters():
    p.requires_grad_(False)
print(f'\n✅ Teacher ready best val=(best_t_acc:.4f) → {TEACHER_PATH}')

```

5b — Distill student (KD + MixUp + EMA + warmup-cosine)

Hard loss: label-smoothed CE (with MixUp). Soft loss: KL between teacher/student softened logits.

```

print('► Student distillation started', flush=True)

class ModelEMA:
    def __init__(self, model, decay=0.999):
        self.ema = copy.deepcopy(model).eval()
        for p in self.ema.parameters():
            p.requires_grad_(False)
        self.decay = decay
    @torch.no_grad()
    def update(self, model):
        d = self.decay
        msd = model.state_dict()
        for k, v in self.ema.state_dict().items():
            if v.dtype.is_floating_point:
                v.mul_(d).add_(msd[k].detach(), alpha=1.0 - d)
            else:
                v.copy_(msd[k])

def mixup_batch(x, y, alpha):
    if alpha <= 0:
        return x, y, 1.0
    lam = np.random.beta(alpha, alpha)
    idx = torch.randperm(x.size(0), device=x.device)
    return lam * x + (1.0 - lam) * x[idx], y, y[idx], lam

def kd_loss(student_logits, teacher_logits, y_a, y_b, lam, class_w):
    # hard (mixup) CE
    log_p = F.log_softmax(student_logits, dim=1)
    ce_a = F.nll_loss(log_p, y_a, weight=class_w, reduction='none')
    ce_b = F.nll_loss(log_p, y_b, weight=class_w, reduction='none')
    hard = (lam * ce_a + (1.0 - lam) * ce_b).mean()
    # soft KD
    T = KD_TEMPERATURE
    soft = F.kl_div(
        F.log_softmax(student_logits / T, dim=1),
        F.log_softmax(teacher_logits / T, dim=1),
        reduction='batchmean',
    ) * (T * T)
    return KD_ALPHA * soft + (1.0 - KD_ALPHA) * hard

```

```

def lr_at_epoch(epoch):
    """Linear warmup then cosine decay."""
    if epoch < WARMUP_EPOCHS:
        return LR_STUDENT + float(epoch + 1) / float(WARMUP_EPOCHS)
    progress = (epoch - WARMUP_EPOCHS) / max(1, STUDENT_EPOCHS - WARMUP_EPOCHS)
    return 1e-6 + 0.5 * (LR_STUDENT - 1e-6) * (1.0 + math.cos(math.pi * progress))

opt_s = torch.optim.AdamW(student.parameters(), lr=LR_STUDENT, weight_decay=WEIGHT_DECAY)
ema = ModelEMA(student, decay=EMA_DECAY)
ce_eval = nn.CrossEntropyLoss(weight=class_weights)

best_val_acc = 0.0
patience_counter = 0

print('=' * 65)
print(f'Student KD   epochs={STUDENT_EPOCHS}   lr={LR_STUDENT}   α_KD={KD_ALPHA}   MixUp={MIXUP_ALPHA}')
print('=' * 65)

for epoch in range(STUDENT_EPOCHS):
    lr = lr_at_epoch(epoch)
    for g in opt_s.param_groups:
        g['lr'] = lr

    student.train()
    total_loss, correct, total = 0.0, 0, 0
    for images, labels in train_loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        mixed, y_a, y_b, lam = mixup_batch(images, labels, MIXUP_ALPHA)
        opt_s.zero_grad()
        s_logits = student(mixed)
        with torch.no_grad():
            t_logits = teacher(mixed)
        loss = kd_loss(s_logits, t_logits, y_a, y_b, lam, class_weights)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(student.parameters(), 5.0)
        opt_s.step()
        ema.update(student)

    total_loss += loss.item() * len(images)
    # accuracy on non-mixed labels is noisy under MixUp - use raw student preds vs y_a
    correct += (s_logits.argmax(1) == y_a).sum().item()
    total += len(images)

    tr_acc = correct / total
    _, va_acc, per_class = evaluate(ema.ema, val_loader, ce_eval)
    marker = ''
    if va_acc > best_val_acc:
        best_val_acc = va_acc
        patience_counter = 0
        torch.save(student.state_dict(), SAVE_PATH)
        torch.save(ema.ema.state_dict(), EMA_PATH)
        marker = ' ← saved EMA'
    else:
        patience_counter += 1
        marker = f' (patience {patience_counter}/{PATIENCE})'

    torch.save({
        'epoch': epoch,
        'student': student.state_dict(),
        'ema': ema.ema.state_dict(),
        'best_val_acc': best_val_acc,
    }, CHECKPOINT_PATH)

    print(f' Epoch {epoch+1:3d}/{STUDENT_EPOCHS} | lr {lr:.2e} | train-{tr_acc:.3f} | '
          f'val(EMA) {va_acc:.3f} | leaf={per_class["leaf"]:.3f} '
          f'not_leaf={per_class["not_leaf"]:.3f}{marker}')

    if patience_counter >= PATIENCE:
        print(f'\n Early stopping at epoch {epoch+1}')
        break

# Prefer EMA for eval / export
student.load_state_dict(torch.load(EMA_PATH, map_location=DEVICE))
print(f'\nBest EMA val accuracy: {best_val_acc:.4f}')
print(f'Loaded EMA weights from {EMA_PATH}')

```

► Student distillation started

Student KD epochs=40 lr=0.001 α_KD=0.7 MixUp=0.2

Epoch	1/40	lr	3.33e-04	train-0.671	val(EMA) 0.529	leaf=1.000 not_leaf=0.000 ← saved EMA
Epoch	2/40	lr	6.67e-04	train-0.695	val(EMA) 0.529	leaf=1.000 not_leaf=0.000 (patience 1/12)
Epoch	3/40	lr	1.00e-03	train-0.728	val(EMA) 0.529	leaf=1.000 not_leaf=0.000 (patience 2/12)
Epoch	4/40	lr	1.00e-03	train-0.694	val(EMA) 0.529	leaf=1.000 not_leaf=0.000 (patience 3/12)
Epoch	5/40	lr	9.98e-04	train-0.709	val(EMA) 0.529	leaf=1.000 not_leaf=0.000 (patience 4/12)
Epoch	6/40	lr	9.93e-04	train-0.719	val(EMA) 0.529	leaf=1.000 not_leaf=0.000 (patience 5/12)
Epoch	7/40	lr	9.84e-04	train-0.733	val(EMA) 0.529	leaf=1.000 not_leaf=0.000 (patience 6/12)
Epoch	8/40	lr	9.71e-04	train-0.708	val(EMA) 0.530	leaf=1.000 not_leaf=0.002 ← saved EMA
Epoch	9/40	lr	9.56e-04	train-0.721	val(EMA) 0.561	leaf=1.000 not_leaf=0.068 ← saved EMA
Epoch	10/40	lr	9.37e-04	train-0.708	val(EMA) 0.707	leaf=1.000 not_leaf=0.377 ← saved EMA
Epoch	11/40	lr	9.14e-04	train-0.725	val(EMA) 0.839	leaf=0.999 not_leaf=0.659 ← saved EMA
Epoch	12/40	lr	8.89e-04	train-0.731	val(EMA) 0.911	leaf=0.998 not_leaf=0.814 ← saved EMA
Epoch	13/40	lr	8.61e-04	train-0.720	val(EMA) 0.950	leaf=0.997 not_leaf=0.896 ← saved EMA
Epoch	14/40	lr	8.31e-04	train-0.707	val(EMA) 0.970	leaf=0.997 not_leaf=0.939 ← saved EMA
Epoch	15/40	lr	7.98e-04	train-0.724	val(EMA) 0.977	leaf=0.996 not_leaf=0.955 ← saved EMA
Epoch	16/40	lr	7.62e-04	train-0.723	val(EMA) 0.983	leaf=0.996 not_leaf=0.969 ← saved EMA
Epoch	17/40	lr	7.25e-04	train-0.719	val(EMA) 0.987	leaf=0.995 not_leaf=0.977 ← saved EMA
Epoch	18/40	lr	6.87e-04	train-0.730	val(EMA) 0.987	leaf=0.994 not_leaf=0.979 (patience 1/12)
Epoch	19/40	lr	6.47e-04	train-0.707	val(EMA) 0.988	leaf=0.993 not_leaf=0.981 ← saved EMA
Epoch	20/40	lr	6.06e-04	train-0.723	val(EMA) 0.988	leaf=0.993 not_leaf=0.982 (patience 1/12)
Epoch	21/40	lr	5.64e-04	train-0.729	val(EMA) 0.989	leaf=0.993 not_leaf=0.984 ← saved EMA
Epoch	22/40	lr	5.22e-04	train-0.731	val(EMA) 0.988	leaf=0.992 not_leaf=0.984 (patience 1/12)
Epoch	23/40	lr	4.79e-04	train-0.716	val(EMA) 0.988	leaf=0.992 not_leaf=0.984 (patience 2/12)
Epoch	24/40	lr	4.37e-04	train-0.733	val(EMA) 0.989	leaf=0.992 not_leaf=0.985 ← saved EMA
Epoch	25/40	lr	3.95e-04	train-0.732	val(EMA) 0.989	leaf=0.993 not_leaf=0.986 ← saved EMA
Epoch	26/40	lr	3.54e-04	train-0.731	val(EMA) 0.989	leaf=0.992 not_leaf=0.986 (patience 1/12)
Epoch	27/40	lr	3.14e-04	train-0.710	val(EMA) 0.989	leaf=0.992 not_leaf=0.986 (patience 2/12)
Epoch	28/40	lr	2.76e-04	train-0.760	val(EMA) 0.989	leaf=0.992 not_leaf=0.986 (patience 3/12)
Epoch	29/40	lr	2.39e-04	train-0.728	val(EMA) 0.989	leaf=0.991 not_leaf=0.986 (patience 4/12)
Epoch	30/40	lr	2.03e-04	train-0.714	val(EMA) 0.989	leaf=0.991 not_leaf=0.986 (patience 5/12)
Epoch	31/40	lr	1.70e-04	train-0.722	val(EMA) 0.989	leaf=0.991 not_leaf=0.987 (patience 6/12)
Epoch	32/40	lr	1.40e-04	train-0.753	val(EMA) 0.990	leaf=0.992 not_leaf=0.987 ← saved EMA
Epoch	33/40	lr	1.12e-04	train-0.735	val(EMA) 0.990	leaf=0.992 not_leaf=0.988 ← saved EMA
Epoch	34/40	lr	8.67e-05	train-0.734	val(EMA) 0.990	leaf=0.992 not_leaf=0.988 (patience 1/12)
Epoch	35/40	lr	6.44e-05	train-0.734	val(EMA) 0.991	leaf=0.992 not_leaf=0.989 ← saved EMA
Epoch	36/40	lr	4.53e-05	train-0.725	val(EMA) 0.991	leaf=0.992 not_leaf=0.989 (patience 1/12)
Epoch	37/40	lr	2.95e-05	train-0.734	val(EMA) 0.991	leaf=0.992 not_leaf=0.989 (patience 2/12)
Epoch	38/40	lr	1.71e-05	train-0.704	val(EMA) 0.991	leaf=0.992 not_leaf=0.990 ← saved EMA
Epoch	39/40	lr	8.18e-06	train-0.730	val(EMA) 0.991	leaf=0.992 not_leaf=0.990 (patience 1/12)
Epoch	40/40	lr	2.80e-06	train-0.732	val(EMA) 0.991	leaf=0.992 not_leaf=0.990 (patience 2/12)

Best EMA val accuracy: 0.9910

Loaded EMA weights from /content/drive/MyDrive/leaf_gate_output_alt/aclis_leaf_gate_96x_alt_ema.pth

5c — Quantization-aware training (fake-quant)

A few epochs of QAT so the float graph is already INT8-aware before TFLite conversion. Falls back gracefully if

torch.ao.quantization APIs differ.

```
[1] print('▶ QAT fine-tune started', flush=True)

# Lightweight fake-quant via torch.quantization (eager) on the student.
# If QAT setup fails on this Colab stack, we skip and rely on PTQ (still valid).

qat_ok = False
try:
    import torch.ao.quantization as tq

    student.cpu()
    student.train()
    student.qconfig = tq.get_default_qat_qconfig('qnnpack')
    # Fuse nothing fancy - TinyLeafGate has ReLU6 after BN which fuses poorly in eager.
    # Instead: train with FakeQuant inserted on activations/weights via prepare_qat on a wrapper.
    # Simpler robust approach used below: noise-injection QAT proxy (additive uniform noise
    # scaled like INT8 bins) - works across torch versions without fuse requirements.
    raise RuntimeError('use noise-proxy QAT')
except Exception as e:
    print(f' Eager QAT path skipped ({e}); using INT8-noise proxy QAT.', flush=True)

def int8_noise_forward(module, x):
    """Fake-quant activations to 256 bins with straight-through estimator."""
    if x.dim() == 4:
        xmin = x.amin(dim=(0, 2, 3), keepdim=True)
        xmax = x.amax(dim=(0, 2, 3), keepdim=True)
    else:
        xmin = x.amin(dim=0, keepdim=True)
        xmax = x.amax(dim=0, keepdim=True)
    scale = (xmax - xmin).clamp_min(1e-6) / 255.0
    x_q = torch.round((x - xmin) / scale) * scale + xmin
    # STE: forward = quantized, backward = identity through x
    return x + (x_q - x).detach()

class QATProxy(nn.Module):
    def __init__(self, base):
        super().__init__()
        self.base = base

    def forward(self, x):
        # inject noise after stem and before head
        x = self.base.stem(x)
        if self.training:
            x = int8_noise_forward(self, x)
        x = self.base.blocks(x)
        if self.training:
            x = int8_noise_forward(self, x)
        return self.base.head(x)

student = student.to(DEVICE)
qat_model = QATProxy(student).to(DEVICE)
opt_q = torch.optim.AdamW(qat_model.parameters(), lr=1e-4, weight_decay=WEIGHT_DECAY)
ce_q = nn.CrossEntropyLoss(weight=class_weights, label_smoothing=LABEL_SMOOTH)

print('=' * 65)
print(f'QAT-proxy fine-tune epochs={QAT_EPOCHS} lr=1e-4')
print('=' * 65)

for epoch in range(QAT_EPOCHS):
    qat_model.train()
    total_loss, correct, total = 0.0, 0, 0
    for images, labels in train_loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        opt_q.zero_grad()
        logits = qat_model(images)
        with torch.no_grad():
            t_logits = teacher(images)
        # keep a bit of KD during QAT
        hard = ce_q(logits, labels)
        soft = F.kl_div(
            F.log_softmax(logits / KD_TEMPERATURE, dim=1),
            F.softmax(t_logits / KD_TEMPERATURE, dim=1),
            reduction='batchmean',
        ) * (KD_TEMPERATURE ** 2)
        loss = 0.5 * soft + 0.5 * hard
        loss.backward()
        opt_q.step()
        total_loss += loss.item() * len(images)
        correct += (logits.argmax(1) == labels.sum().item())
        total += len(images)
    tr_acc = correct / total
    # eval without noise
    student.eval()
    , va_acc, per_class = evaluate(student, val_loader, ce_eval)
    print(f' QAT (epoch+1:3d) / {QAT_EPOCHS} | train {tr_acc:.3f} | val {va_acc:.3f} | '
          f'leaf={per_class["leaf"]:.3f} not_leaf={per_class["not_leaf"]:.3f}')

torch.save(student.state_dict(), EMA_PATH)
torch.save(student.state_dict(), SAVE_PATH)
print(f'✅ QAT-proxy done - weights saved to {EMA_PATH}')
```

▶ QAT fine-tune started
Eager QAT path skipped (use noise-proxy QAT); using INT8-noise proxy QAT.

=====

QAT-proxy fine-tune epochs=5 lr=1e-4

=====

QAT	1/5	train 0.963	val 0.989	leaf=0.994 not leaf=0.983
QAT	2/5	train 0.962	val 0.989	leaf=0.994 not leaf=0.983
QAT	3/5	train 0.964	val 0.990	leaf=0.993 not leaf=0.987
QAT	4/5	train 0.964	val 0.989	leaf=0.994 not leaf=0.982
QAT	5/5	train 0.961	val 0.990	leaf=0.993 not leaf=0.987

✅ QAT-proxy done - weights saved to /content/drive/MyDrive/leaf_gate_output_alt/acilis_leaf_gate_96x_alt_ema.pth

6 — Test evaluation (PyTorch FP32)

Clean test (same as baseline) and camera-stress test (blur + lighting).

```
[1] student.load_state_dict(torch.load(EMA_PATH, map_location=DEVICE))
student.to(DEVICE).eval()

te_loss, te_acc, per_class = evaluate(student, test_loader, ce_eval)
st_loss, st_acc, st_per = evaluate(student, stress_loader, ce_eval)

print('=' * 65)
print('ALT model - PyTorch FP32 TEST')
print('=' * 65)
print(f' Clean test accuracy : {te_acc:.4f} ({te_acc*100:.1f}%)')
print(f' Camera-stress accuracy: {st_acc:.4f} ({st_acc*100:.1f}%)')
print(f' Target : ≥{TARGET_ACC*100:.0f}% '
      f'{"PASS ✅" if te_acc >= TARGET_ACC else "below target"}')
print(' Clean per-class:')
for cls, acc in per_class.items():
    print(f' {cls:<10} {acc:.3f} ({acc*100:.1f}%)')
print(' Stress per-class:')
for cls, acc in st_per.items():
    print(f' {cls:<10} {acc:.3f} ({acc*100:.1f}%)')
```

```
print(f' Weights : {EMA_PATH}')
print('=' * 65)
```

```
=====
ALT model - PyTorch FP32 TEST
=====
```

```
Clean test accuracy : 0.9905 (99.0%)
Camera-stress accuracy: 0.9891 (98.9%)
Target : ≥92% PASS ✓
Clean per-class:
  leaf 0.993 (99.3%)
  not_leaf 0.988 (98.8%)
Stress per-class:
  leaf 0.995 (99.5%)
  not_leaf 0.982 (98.2%)
Weights : /content/drive/MyDrive/leaf_gate_output_alt/aclis_leaf_gate_96x_alt_ema.pth
=====
```

7 — Export TinyEngine-friendly INT8 TFLite

Same Keras twin / GAP(keepdims)+1x1 Conv path as baseline so TinyEngine codegen still works.

```
[1] print('▶ TinyEngine-friendly export started', flush=True)

import numpy as np
import tensorflow as tf

student.load_state_dict(torch.load(EMA_PATH, map_location='cpu'))
student.cpu().eval()
sd = student.state_dict()
model = student # alias used by export / eval cells

def pt_conv_to_keras(w):
    return w.detach().cpu().numpy().transpose(2, 3, 1, 0)

def pt_dw_to_keras(w):
    return w.detach().cpu().numpy().transpose(2, 3, 0, 1)

def pt_linear_to_conv1x1(w):
    arr = w.detach().cpu().numpy().T
    return arr.reshape(1, 1, arr.shape[0], arr.shape[1])

def keras_bn_weights(prefix):
    return [
        sd[f'{prefix}.weight'].detach().cpu().numpy(),
        sd[f'{prefix}.bias'].detach().cpu().numpy(),
        sd[f'{prefix}.running_mean'].detach().cpu().numpy(),
        sd[f'{prefix}.running_var'].detach().cpu().numpy(),
    ]

def pt_matched_bn(name):
    return tf.keras.layers.BatchNormalization(epsilon=1e-5, momentum=0.9, name=name)

def conv_pt_pad(x, layer, name):
    x = tf.keras.layers.ZeroPadding2D(1, name=f'{name}_pad')(x)
    return layer(x)

def build_keras_twin():
    def ds_block(x, cout, stride, name):
        x = conv_pt_pad(
            x,
            tf.keras.layers.DepthwiseConv2D(
                3, strides=stride, padding='valid', use_bias=False, name=f'{name}_dw'),
            f'{name}_dw',
        )
        x = pt_matched_bn(f'{name}.bn1')(x)
        x = tf.keras.layers.ReLU(max_value=6.0, name=f'{name}_relu')(x)
        x = tf.keras.layers.Conv2D(cout, 1, use_bias=False, name=f'{name}_pw')(x)
        x = pt_matched_bn(f'{name}.bn2')(x)
        x = tf.keras.layers.ReLU(max_value=6.0, name=f'{name}_relu2')(x)
        return x

    inp = tf.keras.Input(shape=(IMAGE_SIZE, IMAGE_SIZE, 3), name='input')
    x = conv_pt_pad(
        inp,
        tf.keras.layers.Conv2D(
            32, 3, strides=2, padding='valid', use_bias=False, name='stem_conv'),
        'stem',
    )
    x = pt_matched_bn('stem.bn')(x)
    x = tf.keras.layers.ReLU(max_value=6.0, name='stem_relu')(x)
    x = ds_block(x, 48, 2, 'b0')
    x = ds_block(x, 64, 2, 'b1')
    x = ds_block(x, 96, 2, 'b2')
    x = ds_block(x, 128, 1, 'b3')
    x = tf.keras.layers.GlobalAveragePooling2D(keepdims=True, name='gap')(x)
    x = tf.keras.layers.Conv2D(NUM_CLASSES, 1, use_bias=True, name='cls_conv')(x)
    out = tf.keras.layers.Reshape((NUM_CLASSES,)), name='output')(x)
    return tf.keras.Model(inp, out, name='TinyLeafGate_ALT_TE')

kmodel = build_keras_twin()
kmodel.get_layer('stem_conv').set_weights([pt_conv_to_keras(sd['stem.0.weight'])])
kmodel.get_layer('stem.bn').set_weights(keras_bn_weights('stem.1'))

for kname, ptname in [('b0', 'blocks.0'), ('b1', 'blocks.1'),
                      ('b2', 'blocks.2'), ('b3', 'blocks.3')]:
    kmodel.get_layer(f'{kname}_dw').set_weights([pt_dw_to_keras(sd[f'{ptname}.dw.weight'])])
    kmodel.get_layer(f'{kname}.bn1').set_weights(keras_bn_weights(f'{ptname}.bn1'))
    kmodel.get_layer(f'{kname}_pw').set_weights([pt_conv_to_keras(sd[f'{ptname}.pw.weight'])])
    kmodel.get_layer(f'{kname}.bn2').set_weights(keras_bn_weights(f'{ptname}.bn2'))

kmodel.get_layer('cls_conv').set_weights([
    pt_linear_to_conv1x1(sd['head.3.weight']),
    sd['head.3.bias'].detach().cpu().numpy(),
])

print('✓ Keras twin built and weights copied')
kmodel.summary()

kmodel.trainable = False
n_check = min(16, len(val_dataset))
diffs = []
for i in range(n_check):
    img, _ = val_dataset[i]
    with torch.no_grad():
        pt_out = model(img.unsqueeze(0)).numpy().reshape(-1)
        x = img.numpy().transpose(1, 2, 0)[None, ...].astype(np.float32)
        k_out = kmodel.predict(x, verbose=0).reshape(-1)
        diffs.append(np.max(np.abs(pt_out - k_out)))

print(f'Float max|PT-Keras| over {n_check} images: mean={np.mean(diffs):.5f} max={np.max(diffs):.5f}')
if np.max(diffs) > 0.01:
    raise RuntimeError(
        f'PT-Keras logit mismatch too large (max={np.max(diffs):.5f}). Do not export.'
    )
print('✓ PT - Keras logits match')
```

b0_relu1 (ReLU)	(None, 24, 24, 32)	0
b0_pw (Conv2D)	(None, 24, 24, 48)	1,536
b0_bn2 (BatchNormalization)	(None, 24, 24, 48)	192
b0_relu2 (ReLU)	(None, 24, 24, 48)	0
b1_dw_pad (ZeroPadding2D)	(None, 26, 26, 48)	0
b1_dw (DepthwiseConv2D)	(None, 12, 12, 48)	432
b1_bn1 (BatchNormalization)	(None, 12, 12, 48)	192
b1_relu1 (ReLU)	(None, 12, 12, 48)	0
b1_pw (Conv2D)	(None, 12, 12, 64)	3,072
b1_bn2 (BatchNormalization)	(None, 12, 12, 64)	256
b1_relu2 (ReLU)	(None, 12, 12, 64)	0
b2_dw_pad (ZeroPadding2D)	(None, 14, 14, 64)	0
b2_dw (DepthwiseConv2D)	(None, 6, 6, 64)	576
b2_bn1 (BatchNormalization)	(None, 6, 6, 64)	256
b2_relu1 (ReLU)	(None, 6, 6, 64)	0
b2_pw (Conv2D)	(None, 6, 6, 96)	6,144
b2_bn2 (BatchNormalization)	(None, 6, 6, 96)	384
b2_relu2 (ReLU)	(None, 6, 6, 96)	0
b3_dw_pad (ZeroPadding2D)	(None, 8, 8, 96)	0
b3_dw (DepthwiseConv2D)	(None, 6, 6, 96)	864
b3_bn1 (BatchNormalization)	(None, 6, 6, 96)	384
b3_relu1 (ReLU)	(None, 6, 6, 96)	0
b3_pw (Conv2D)	(None, 6, 6, 128)	12,288
b3_bn2 (BatchNormalization)	(None, 6, 6, 128)	512
b3_relu2 (ReLU)	(None, 6, 6, 128)	0
gap (GlobalAveragePooling2D)	(None, 1, 1, 128)	0
cls_conv (Conv2D)	(None, 1, 1, 2)	256
output (Reshape)	(None, 2)	0

Total params: 28,754 (112.32 KB)
Trainable params: 27,538 (107.57 KB)
Non-trainable params: 1,216 (4.75 KB)
Float max|PT-Keras| over 16 images: mean=0.00000 max=0.00000
✔ PT → Keras logits match

```
print('▶ INT8 TFLite conversion', flush=True)

calib_ds = val_dataset

def make_rep_data():
    n = min(200, len(calib_ds))
    for i in range(n):
        img, _ = calib_ds[i]
        x = img.numpy().transpose(1, 2, 0)[None, ...].astype(np.float32)
        yield [x]

TF_SAVED = '/content/aclis_leaf_gate_alt_tf'
!rm -rf {TF_SAVED}
kmodel.export(TF_SAVED)

converter = tf.lite.TFLiteConverter.from_keras_model(kmodel)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = make_rep_data
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.int8
converter.inference_output_type = tf.int8
tflite_model = converter.convert()

os.makedirs(os.path.dirname(TFLITE_PATH) or '.', exist_ok=True)
with open(TFLITE_PATH, 'wb') as f:
    f.write(tflite_model)

tflite_kb = os.path.getsize(TFLITE_PATH) / 1024.0
print(f'✔ INT8 TFLite saved: (TFLITE_PATH)')
print(f'File size: {tflite_kb:.1f} KB')
```

```
* Endpoint 'serve'
args 0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 96, 96, 3), dtype=tf.float32, name='input')
Output Type:
TensorSpec(shape=(None, 2), dtype=tf.float32, name=None)
Captures:
137041204572240: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896410448: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896409872: TensorSpec(shape=(), dtype=tf.resource, name=None)
137041204576080: TensorSpec(shape=(), dtype=tf.resource, name=None)
137041204575696: TensorSpec(shape=(), dtype=tf.resource, name=None)
137041204575504: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896410256: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896410064: TensorSpec(shape=(), dtype=tf.resource, name=None)
137041204572048: TensorSpec(shape=(), dtype=tf.resource, name=None)
137041204573968: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896412176: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896414480: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896414672: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896413136: TensorSpec(shape=(), dtype=tf.resource, name=None)
13704089641312: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896415056: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896415248: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896414288: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896414864: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896415824: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896413520: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896416976: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896417168: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896416408: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896413328: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896416016: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896415632: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896415440: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896416592: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896418128: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896417552: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896417744: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896417936: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896416704: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896419088: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896418512: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896418704: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896418896: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896412752: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896420048: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896420432: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896420624: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

```
137040896419664: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896420240: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896421200: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896421584: TensorSpec(shape=(), dtype=tf.resource, name=None)
137040896421392: TensorSpec(shape=(), dtype=tf.resource, name=None)
/usr/local/lib/python3.12/dist-packages/tensorflow/lite/python/convert.py:863: UserWarning: Statistics for quantized inputs were expected, but not specified;
warnings.warn(
✔ INT8 TFLite saved: /content/drive/MyDrive/leaf_gate_output_alt/aclis_leaf_gate_96x_alt_full_int8.tflite
File size: 51.1 KB
```

```
print('▶ Validate TFLite graph for TinyEngine', flush=True)

from tensorflow.lite.python import schema_py_generated as schema_fb

interp = tf.lite.Interpreter(model_path=TFLITE_PATH)
interp.allocate_tensors()
inp = interp.get_input_details()[0]
out = interp.get_output_details()[0]
print('Input:', list(inp['shape']), inp['dtype'], 'quant=', inp.get('quantization'))
print('Output:', list(out['shape']), out['dtype'], 'quant=', out.get('quantization'))

_OP_NAMES = {
    getattr(schema_fb.BuiltinOperator, k): k
    for k in dir(schema_fb.BuiltinOperator)
    if k.isupper() and isinstance(getattr(schema_fb.BuiltinOperator, k), int)
}

buf = open(TFLITE_PATH, 'rb').read()
fb_model = schema_fb.Model.GetRootAsModel(buf, 0)
g = fb_model.Subgraphs(0)

ops, bad = [], []
for i in range(g.OperatorsLength()):
    op = g.Operators(i)
    oc = fb_model.OperatorCodes(op.OpcodeIndex())
    code = oc.BuiltinCode()
    if code == 0 and hasattr(oc, 'DeprecatedBuiltinCode'):
        try:
            dep = oc.DeprecatedBuiltinCode()
            if dep:
                code = dep
        except Exception:
            pass
    name = _OP_NAMES.get(code, f'OP_{code}')
    ops.append(name)
    if name == 'RESHAPE':
        out_ti = op.Outputs(0)
        t = g.Tensors(out_ti)
        shape = [int(t.Shape(j)) for j in range(t.ShapeLength())]
        if shape in ([1, 1], [1]):
            bad.append(f'RESHAPE - {shape}')
        else:
            print(f' reshape ok: {shape}')

print('\n\nOperators:')
for i, n in enumerate(ops):
    print(f' {i:02d} {n}')

has_pool = any(n in ('MEAN', 'AVERAGE_POOL_2D') for n in ops)
has_cls = any(n in ('CONV_2D', 'FULLY_CONNECTED') for n in ops)

x = np.zeros(tuple(inp['shape']), dtype=np.int8)
interp.set_tensor(inp['index'], x)
interp.invoke()
print('Smoke output:', interp.get_tensor(out['index']))

if bad or not has_pool or not has_cls:
    raise RuntimeError(f'Graph not TinyEngine-friendly: bad={bad} pool={has_pool} cls={has_cls}')
print('\n✔ TFLite looks TinyEngine-friendly')
print('Download:', TFLITE_PATH)
```

```
▶ Validate TFLite graph for TinyEngine
Input : [np.int32(1), np.int32(96), np.int32(96), np.int32(3)] <class 'numpy.int8'> quant= (0.01845697872340679, -16)
Output: [np.int32(1), np.int32(2)] <class 'numpy.int8'> quant= (0.018627634271979332, -1)
reshape ok: [1, 2]
\nOperators:
00 PAD
01 CONV_2D
02 PAD
03 DEPTHWISE_CONV_2D
04 CONV_2D
05 PAD
06 DEPTHWISE_CONV_2D
07 CONV_2D
08 PAD
09 DEPTHWISE_CONV_2D
10 CONV_2D
11 DEPTHWISE_CONV_2D
12 CONV_2D
13 MEAN
14 CONV_2D
15 SHAPE
16 STRIDED_SLICE
17 PACK
18 RESHAPE
Smoke output: [[-84 57]]
\n✔ TFLite looks TinyEngine-friendly
Download: /content/drive/MyDrive/leaf_gate_output_alt/aclis_leaf_gate_96x_alt_full_int8.tflite
/usr/local/lib/python3.12/dist-packages/tensorflow/lite/python/interpreter.py:457: UserWarning: Warning: tf.lite.Interpreter is deprecated and is scheduled
TF 2.20. Please use the LiteRT interpreter from the ai edge litert package.
See the [migration guide](https://ai.google.dev/edge/litert/migration)
for details.

warnings.warn(_INTERPRETER_DELETION_WARNING)
```

7b — INT8 TFLite test (alt model alone)

```
print('▶ INT8 TFLite full test-set evaluation (ALT)', flush=True)

interp = tf.lite.Interpreter(model_path=TFLITE_PATH)
interp.allocate_tensors()
inp = interp.get_input_details()[0]
out = interp.get_output_details()[0]
in_scale, in_zp = inp['quantization']
assert in_scale > 0

def quantize_nhw(cw float: np.ndarray) -> np.ndarray:
    x = cw.float.transpose(1, 2, 0).astype(np.float32)
    v = x / float(in_scale)
    v = v + np.where(v >= 0.0, 0.5, -0.5)
    q = np.trunc(v).astype(np.int32) + int(in_zp)
    return np.clip(q, -128, 127).astype(np.int8)

def eval_tflite_on_dataset(ds, tag):
    correct = 0
    total = 0
    class_correct = [0] * NUM_CLASSES
    class_total = [0] * NUM_CLASSES
```

```

agree_pt = 0
model.cpu().eval()
for i in range(len(ds)):
    img, label = ds[i]
    x_int8 = quantize_nhw(img.numpy())
    interp.set_tensor(inp['index'], x_int8[None, ...])
    interp.invoke()
    y = interp.get_tensor(out['index']).reshape(-1)
    pred = int(np.argmax(y))
    with torch.no_grad():
        pt_pred = int(model(img.unsqueeze(0)).argmax(1).item())
    correct += int(pred == label)
    agree_pt += int(pred == pt_pred)
    total += 1
    class_total[label] += 1
    class_correct[label] += int(pred == label)
    if (i + 1) % 500 == 0 or (i + 1) == total:
        print(f' [{tag}] - {i+1}/{len(ds)}', flush=True)
acc = correct / total
per = {CLASSES[i]: (class_correct[i] / class_total[i] if class_total[i] else 0.0)
        for i in range(NUM_CLASSES)}
return acc, agree_pt / total, per

int8_acc, pt_agree, int8_per = eval_tflite_on_dataset(test_dataset, 'clean')
int8_stress, _, int8_stress_per = eval_tflite_on_dataset(stress_dataset, 'stress')
gap = abs(int8_acc - te_acc)

print('=' * 65)
print('ALT INT8 TFLite TEST')
print('=' * 65)
print(f' Clean INT8 accuracy : {int8_acc:.4f} ({int8_acc*100:.1f}%)')
print(f' Clean PyTorch FP32 : {te_acc:.4f} ({te_acc*100:.1f}%)')
print(f' |Δ| accuracy : {gap:.4f}')
print(f' Pred agreement PT : {pt_agree:.4f}')
print(f' Camera-stress INT8 : {int8_stress:.4f} ({int8_stress*100:.1f}%)')
for cls in CLASSES:
    print(f' - clean {cls:<10} {int8_per[cls]:.3f} stress {int8_stress_per[cls]:.3f}')
print(f' Artifact : {TFLITE_PATH}')
print('=' * 65)

if gap > 0.01:
    raise RuntimeError(
        f'INT8 test acc ({int8_acc:.4f}) differs from PyTorch ({te_acc:.4f}) by {gap:.4f} > 1%.'
    )
print('✅ INT8 == PyTorch clean test accuracy within 1%')

```

```

[stress] - 3629/3673
[stress] - 3630/3673
[stress] - 3631/3673
[stress] - 3632/3673
[stress] - 3633/3673
[stress] - 3634/3673
[stress] - 3635/3673
[stress] - 3636/3673
[stress] - 3637/3673
[stress] - 3638/3673
[stress] - 3639/3673
[stress] - 3640/3673
[stress] - 3641/3673
[stress] - 3642/3673
[stress] - 3643/3673
[stress] - 3644/3673
[stress] - 3645/3673
[stress] - 3646/3673
[stress] - 3647/3673
[stress] - 3648/3673
[stress] - 3649/3673
[stress] - 3650/3673
[stress] - 3651/3673
[stress] - 3652/3673
[stress] - 3653/3673
[stress] - 3654/3673
[stress] - 3655/3673
[stress] - 3656/3673
[stress] - 3657/3673
[stress] - 3658/3673
[stress] - 3659/3673
[stress] - 3660/3673
[stress] - 3661/3673
[stress] - 3662/3673
[stress] - 3663/3673
[stress] - 3664/3673
[stress] - 3665/3673
[stress] - 3666/3673
[stress] - 3667/3673
[stress] - 3668/3673
[stress] - 3669/3673
[stress] - 3670/3673
[stress] - 3671/3673
[stress] - 3672/3673
[stress] - 3673/3673

=====
ALT INT8 TFLite TEST
=====
Clean INT8 accuracy : 0.9987 (99.1%)
Clean PyTorch FP32 : 0.9985 (99.0%)
|Δ| accuracy : 0.0003
Pred agreement PT : 0.9992
Camera-stress INT8 : 0.9886 (98.9%)
clean leaf 0.992 stress 0.995
clean not leaf 0.989 stress 0.982
Artifact : /content/drive/MyDrive/leaf_gate_output/alt/aclis_leaf_gate_96x_alt_full_int8.tflite
=====
✅ INT8 == PyTorch clean test accuracy within 1%

```

8 — Side-by-side: baseline TFLite vs alt TFLite

Loads the **current deployed** INT8 model and this notebook's INT8 model. Same clean test + camera-stress set. This is the comparison you want before deciding what to flash onto the STM32.

Upload / keep baseline at:

MyDrive/leaf_gate_output/aclis_leaf_gate_96x_full_int8.tflite

```

print('▶ Baseline vs ALT comparison', flush=True)

def run_tflite(path, ds):
    it = tf.lite.Interpreter(model_path=path)
    it.allocate_tensors()
    ind = it.get_input_details()[0]
    oud = it.get_output_details()[0]
    scale, zp = ind['quantization']
    assert scale > 0
    correct = 0
    total = 0
    class_correct = [0] * NUM_CLASSES
    class_total = [0] * NUM_CLASSES
    # confusion-m: false leaf (pred leaf, true not) and false reject (pred not, true leaf)
    false_leaf = 0
    false_reject = 0
    for i in range(len(ds)):

```

```

img, label = ds[1]
x = img.numpy().transpose(1, 2, 0).astype(np.float32)
v = x / float(scale)
v = v + np.where(v >= 0.0, 0.5, -0.5)
q = np.clip(np.trunc(v).astype(np.int32) + int(zp), -128, 127).astype(np.int8)
it.set_tensor(ind['index'], q[None, ...])
it.invoke()
pred = int(np.argmax(it.get_tensor(out['index']).reshape(-1)))
correct += int(pred == label)
total += 1
class_total[label] += 1
class_correct[label] += int(pred == label)
if label == 1 and pred == 0: # true not leaf, predicted leaf
    false_leaf += 1
if label == 0 and pred == 1: # true leaf, predicted not leaf
    false_reject += 1
acc = correct / total
per = {CLASSES[i]: class_correct[i] / class_total[i] if class_total[i] else 0.0
        for i in range(NUM_CLASSES)}
return {
    'acc': acc,
    'per': per,
    'false_leaf': false_leaf,
    'false_reject': false_reject,
    'n': total,
    'kb': os.path.getsize(path) / 1024.0,
}

results = {'ALT': {}}
results['ALT']['clean'] = run_tflite(TFLITE_PATH, test_dataset)
results['ALT']['stress'] = run_tflite(TFLITE_PATH, stress_dataset)

has_baseline = os.path.isfile(BASELINE_TFLITE)
if has_baseline:
    results['BASELINE'] = {}
    results['BASELINE']['clean'] = run_tflite(BASELINE_TFLITE, test_dataset)
    results['BASELINE']['stress'] = run_tflite(BASELINE_TFLITE, stress_dataset)
else:
    print(f'⚠ Baseline TFLite not found at {BASELINE_TFLITE}')
    print('⚠ Skipping baseline columns – re-run after copying the current model to Drive.')

def row(name, split):
    r = results[name][split]
    return (f'{r["acc"]*100:5.2f}%',
            f'{r["per"]["leaf"]*100:5.1f}%',
            f'{r["per"]["not leaf"]*100:5.1f}%',
            f'{r["false_reject"]*100:5.1f}%',
            f'{r["false_reject"]*100:5.1f}%',
            f'{r["false_reject"]*100:5.1f}%',
            f'{r["kb"]*1024:5.1f}')

print()
print('=' * 88)
print('REAL-LIFE PROXY COMPARISON (clean Kaggle test | camera-stress = blur+lighting)')
print('=' * 88)
hdr = f'{<10} {"split":<8} {"acc":>7} {"leaf":>7} {"not leaf":>9} {"FN leaf":>8} {"FP leaf":>8} {"KB":>6}'
print(hdr)
print('=' * 88)

order = (('BASELINE', 'ALT') if has_baseline else ('ALT'))
for name in order:
    for split in ('clean', 'stress'):
        acc, leaf, nol, fn, fp, kb = row(name, split)
        print(f'{name:<10} {split:<8} {acc:>7} {leaf:>7} {nol:>9} {fn:>8} {fp:>8} {kb:>6}')

print('=' * 88)
print('FN leaf = true leaf predicted not leaf (gate rejects a real leaf)')
print('FP leaf = true not leaf predicted leaf (disease CNN runs on junk)')
print()

if has_baseline:
    b, a = results['BASELINE'], results['ALT']
    print('Deltas (ALT – BASELINE), positive = alt better accuracy / fewer errors:')
    for split in ('clean', 'stress'):
        d_acc = (a[split]['acc'] - b[split]['acc']) * 100
        d_fn = b[split]['false_reject'] - a[split]['false_reject']
        d_fp = b[split]['false_reject'] - a[split]['false_reject']
        print(f' {split} Δacc={d_acc:+.2f} pts ΔFN_leaf={d_fn:+d} ΔFP_leaf={d_fp:+d}')

    # Pick recommendation: prefer stress (closer to field), break ties on clean
    prefer_alt = (
        a['stress']['acc'] > b['stress']['acc'] + 0.002
        or (abs(a['stress']['acc'] - b['stress']['acc']) <= 0.002
            and a['clean']['acc'] >= b['clean']['acc'])
    )
    print()
    if prefer_alt:
        print('💡 Recommendation: prefer ALT for field/camera robustness '
              f'({TFLITE_PATH})')
    else:
        print('💡 Recommendation: keep BASELINE for now '
              f'({BASELINE_TFLITE})')
    print('=' * 88)

> Baseline vs ALT comparison
=====
REAL-LIFE PROXY COMPARISON (clean Kaggle test | camera-stress = blur+lighting)
=====
model    split    acc    leaf  not leaf  FN leaf  FP leaf  KB
-----
BASELINE clean   99.13%  99.4%   98.8%    11      21     50.9
BASELINE stress  97.63%  99.1%   96.0%    17      70     50.9
ALT      clean   99.07%  99.2%   98.9%    15      19     51.1
ALT      stress  98.86%  99.5%   98.2%    10      32     51.1
=====
FN leaf = true leaf predicted not leaf (gate rejects a real leaf)
FP leaf = true not leaf predicted leaf (disease CNN runs on junk)

Deltas (ALT – BASELINE), positive = alt better accuracy / fewer errors:
[clean] Δacc=-0.05 pts ΔFN_leaf=-4 ΔFP_leaf=+2
[stress] Δacc=+1.23 pts ΔFN_leaf=+7 ΔFP_leaf=+38

💡 Recommendation: prefer ALT for field/camera robustness (/content/drive/MyDrive/leaf_gate_output_alt/aclis_leaf_gate_96x_alt_full_int8.tflite)
=====

```

9 — STM32 budget check + next steps

Alt TFLite should be same size as baseline (50 KB). To deploy the winner:

- Copy chosen *_full_int8.tflite into `Ikmal/Leaf Gate Model/` (rename to the path `codegen_leaf_gate_c.py` expects, or point the emitter at the alt file).
- Run `../tinycnn/venv/bin/python codegen_leaf_gate_c.py`
- Rebuild / flash ACLIS; compare on-device with real camera frames (the true real-life test).

On-device A/B tip: flash baseline, capture 20–30 leaf + 20–30 non-leaf field shots and log gate decisions; then flash alt and repeat the same shots.

```
DISEASE_FLASH = 723
FLASH_BUDGET = 1024
alt_kb = os.path.getsize(TFLITE_PATH) / 1024.0
base_kb = os.path.getsize(BASELINE_TFLITE) / 1024.0 if os.path.isfile(BASELINE_TFLITE) else float('nan')

print('=' * 65)
print('Flash picture (weights only)')
print('=' * 65)
print(f' Disease model      : {DISEASE_FLASH} KB')
print(f' Baseline gate       : {base_kb:.1f} KB')
print(f' ALT gate             : {alt_kb:.1f} KB')
print(f' Disease + ALT        : {DISEASE_FLASH + alt_kb:.1f} / {FLASH_BUDGET} KB '
      f'({"FITS" if DISEASE_FLASH + alt_kb <= FLASH_BUDGET else "OVER"})')
print('=' * 65)
print(f'\nAlt artifact: {TFLITE_PATH}')
```

Flash picture (weights only)

```
=====
Disease model      : 723 KB
Baseline gate      : 50.9 KB
ALT gate           : 51.1 KB
Disease + ALT      : 774.1 / 1024 KB (FITS)
```

\nAlt artifact: /content/drive/MyDrive/leaf_gate_output_alt/aclis_leaf_gate_96x_alt_full_int8.tflite

Colab paid products - Cancel contracts here

Variables Terminal

2:18 PM